Deep Q-Learning for Optimal Execution in Financial Markets

Matthew Millward

Durham University, Supervised by Marija Zamaklar

View code here

What is Reinforcement Learning?

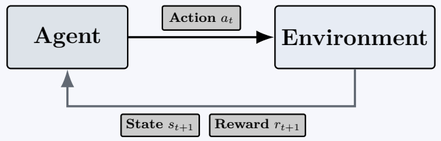
Humans use reinforcement learning every day. Consider a child learning to ride a bicycle. No amount of instructions can replace the experience of balancing, pedalling, and adjusting to maintain stability. Instead, the child tries different actions, learns from their mistakes, and gradually improves through repeated attempts.

Unlike supervised learning, where a teacher provides correct answers, reinforcement learning is defined by interaction [1], and is structured around four essential elements:

- **Agent:** The learner or decision maker (the child),
- **Environment:** Everything external to the agent (the bicycle, terrain, gravity),
- **Actions:** The choices the agent can make (pedalling, steering, braking),
- **Rewards:** The feedback signal (staying balanced vs. falling off).

The Agent-Environment Loop

At each moment in time, the agent observes the current **state** s_t (e.g., the bicycle's speed and angle), selects an **action** a_t (e.g., pedal faster), and receives feedback in the form of a **reward** r_{t+1} and new state s_{t+1}.



This interaction repeats, creating a **trajectory**: a sequence of states, actions, and rewards over time. For the bicycle example, each trajectory continues until the child either falls (ending that attempt) or successfully rides to their destination.

$$s_t \xrightarrow{a_t} s_{t+1}, r_{t+1} \xrightarrow{a_{t+1}} s_{t+2}, r_{t+2} \xrightarrow{a_{t+2}} \cdots$$

The agent's **goal** is to find the best decision-making strategy: one that maximises the total reward collected over time. An agent's strategy is called a **policy** π, and specifies which action to select in each state. Through experience, the policy improves: the child falls less often and rides more confidently, earning higher cumulative rewards.

Maximising Rewards and Value Functions

However, future rewards are uncertain; the environment may change unexpectedly or the child might fall at any moment. To account for this, we introduce a **discounted cumulative return** G_t [1], which prioritises immediate rewards over future ones:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{i=0}^{\infty} \gamma^i r_{i+t+1},$$

where $\gamma \in [0, 1]$ is the discount factor. Since a single trajectory may end in success or failure depending on chance, we evaluate a policy by averaging the return over all possible trajectories from an initial state (or action). A **value function** captures this:

- **State-value**: $V_\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$,
- **Action-value**: $Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$.

Calculating the total expected return over all possible trajectories is difficult. Instead, we break it down into two parts: the immediate reward from the current action, and the value of the next state. This is defined by the **Bellman equation** [1]:

$$Q_\pi(s, a) = \mathbb{E}_\pi[r_{t+1} + \gamma Q_\pi(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a].$$

Tabular Q-Learning

So far we have introduced a way to evaluate how good an action is under a specific policy. But different policies lead to different values and some strategies are simply better than others. The policy with the highest values is called the **optimal policy** π^* [2], and its corresponding **optimal action-values** are Q^*:

$$Q^*(s, a) = \max_\pi Q_\pi(s, a) = Q_{\pi^*}(s, a).$$

If we know Q^*, the agent can pick the highest-valued action in each state to recover the optimal policy. But Q^* is unknown: the agent must discover it through experience.

For small environments, we store our **current estimate** $\hat{Q}(s, a)$ in a table, with one entry per state-action pair, starting at zero. The agent **explores** one step at a time, usually selecting the action it thinks is best, and occasionally tries a random one to avoid missing better paths. After each step, it **compares** what it observed to its previous belief, shifts its estimate towards the truth, then moves to the new state and repeats. This is called **temporal difference** learning [2] and each Q-value estimate is updated as follows:

$$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha \overbrace{\Big[\underbrace{r_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a')}_{\text{Target (Reality + Future Guess)}} - \underbrace{\hat{Q}(s_t, a_t)}_{\text{Old Estimate}} \Big]}^{\text{Temporal Difference Error}},$$

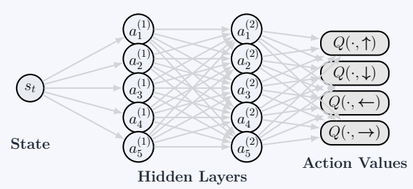
where $\alpha \in [0, 1)$ is the **learning rate**. When one trajectory ends, the agent resets and begins another. After enough episodes, the entries in $\hat{Q}$ stop changing, and at this point $\hat{Q}$ has converged to Q^* and the agent has found the optimal policy.

Deep Q-Networks (DQN)

A table with one entry per state-action pair becomes infeasible for large or continuous environments. Instead, we approximate $Q(s, a)$ using a **Neural Network** [3], labelled $Q_\theta(s, a)$, with parameters θ. The network takes the current state s_t as input and outputs an estimated Q-value for every action. Internally, each layer of the neural network applies a **linear transformation** followed by a **non-linear** activation:

$$\boldsymbol{z} = W\boldsymbol{x} + \boldsymbol{b}, \quad \boldsymbol{a} = \sigma(\boldsymbol{z}),$$

where $\sigma(z)$ is typically $\mathrm{ReLU}(z) = \max\{0, z\}$. By stacking multiple layers, the network can capture non-linear relationships between states and action-values.



The DQN learning process mirrors tabular Q-learning: first observe a single transition, construct a target, and update. The difference is *what* we update; rather than a single table entry, we adjust the shared parameters θ (the weights W and biases $\boldsymbol{b}$ across all layers) so that the prediction moves closer to the target. We minimise:

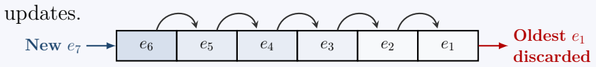
$$\mathcal{L}(\theta) = \frac{1}{2}\Big(\underbrace{r_{t+1} + \gamma \max_{a'} Q_\theta(s_{t+1}, a')}_{\text{Target } y} - \underbrace{Q_\theta(s_t, a_t)}_{\text{Prediction}} \Big)^2,$$

using **gradient descent**: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$, where η is our new learning rate.

The Stability Problem: In supervised learning, targets are fixed (the network knows the true correct answer). In RL, we *estimate* the target using the network's own predictions, so every time θ updates, the target shifts too. This **moving target**, combined with highly correlated sequential data, makes naive DQN training unstable.

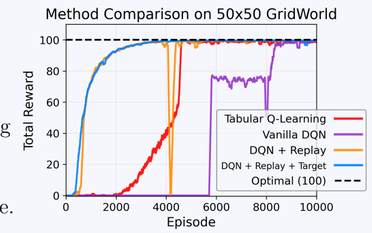
Stabilising DQN

Experience Replay: Consecutive experiences are nearly identical, so training on them directly produces high-variance gradient estimates that overfit to recent data. Instead, we store up to N experiences ($e_i = (s_t, a_t, r_{t+1}, s_{t+1})$) in a **replay buffer** [3] and sample them randomly, mixing old and new experiences for more stable, lower-variance updates.



Target Network: The target depends on the same network we are updating, so it shifts with every improvement. To stabilise this, we freeze a copy of the network [3] to compute targets, and only sync it with the main network every C steps.

Example: The figure to the right compares each Q-learning method on a 50×50 grid navigation task. Standard DQN (purple) is slowest and unstable, and introducing experience replay (orange) speeds up learning but briefly chases an incorrect target (dip at ~4000). Adding the target network (blue) produces the fastest, most stable convergence.



An Application: The Optimal Execution Problem

Large investors often need to sell massive quantities of stock. Selling too fast exhausts buyers, crashing the price (**market impact**). Selling too slowly risks prices drifting unfavourably. The classical solution (Almgren-Chriss) [4] uses stochastic control to derive a schedule that minimises costs, but assumes linear market impact and constant **liquidity** (the availability of willing buyers), which produces a predictable strategy vulnerable to **predatory trading**.

DQN Solution: We propose execution as a reinforcement learning problem:

- **State:** Inventory remaining, time left, current liquidity,
- **Action:** How many shares to sell at this time,
- **Reward:** Negative execution cost (sell at better prices → higher reward).

The DQN agent learns directly from market data with no assumptions about dynamics, learning to pause during liquidity droughts, or accelerate when buyers appear [5].

Conclusions & Future Work

The progression from tabular Q-learning to stabilised DQN has given us a way to approach problems where classical methods struggle due to their assumptions. This is particularly useful in financial settings, allowing us to solve optimal execution problems without relying on unrealistic assumptions. Possible future work includes:

- Adding predatory agents that attempt to detect and exploit the strategy,
- Evaluating performance on real limit order book data.

References

[1] Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction.* MIT Press.

[2] Watkins, C.J.C.H. & Dayan, P. (1992). Q-learning. *Machine Learning.*

[3] Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature.*

[4] Almgren, R. & Chriss, N. (2000). Optimal execution of portfolio transactions. *Journal of Risk.*

[5] Ning, B., Lin, F.H.T. & Jaimungal, S. (2021). Double Deep Q-Learning for Optimal Execution. *Applied Mathematical Finance*, 28(4), 361-380.